Lista de Exercícios – Programação Orientada a Objetos

COMP0496 – Programação A | Turma T01 | 2026.2
Prof. Dr. André Yoshiaki Kashiwabara

Orientações

- Resolva cada questão em Python 3, definindo as classes pedidas com os nomes indicados.
- Teste sua solução com o código de uso fornecido e compare com o resultado esperado.
- Atributos com prefixo `_` são internos: acesse-os somente de dentro da própria classe.
- As questões estão em ordem crescente de dificuldade: [B]ásico, [I]ntermediário e [A]vançado.

Parte 1 – Classes, objetos e atributos

1. Retângulo [B]

Crie a classe `Retangulo`, que recebe `largura` e `altura` no construtor e oferece os métodos `area()`, `perimetro()` e `eh_quadrado()`.

```
r = Retangulo(3, 4)
print(r.area(), r.perimetro(), r.eh_quadrado())
print(Retangulo(5, 5).eh_quadrado())
```

Resultado esperado: `12 14 False` e, na linha seguinte, `True`.

2. Contador de produtos [B]

Crie a classe `Produto` (atributos `nome` e `preco`) com um *atributo de classe* `total_criados`, incrementado sempre que um novo produto é instanciado.

```
Produto("Caneta", 2.5)
Produto("Caderno", 18.0)
p3 = Produto("Lápis", 1.2)
print(Produto.total_criados, p3.total_criados)
```

Resultado esperado: `3 3`. Explique por que `p3.total_criados` também mostra 3, mesmo sem esse atributo ter sido criado no objeto.

3. Encontre o defeito [I]

O código abaixo deveria manter uma lista de alunos separada para cada turma, mas não faz isso.

```
class Turma:
    alunos = []

    def __init__(self, codigo):
        self.codigo = codigo

    def matricular(self, nome):
        self.alunos.append(nome)

t1, t2 = Turma("T01"), Turma("T02")
t1.matricular("Ana")
print(t2.alunos)
```

(a) O que é impresso? Explique o motivo, usando a diferença entre atributo de classe e atributo de instância.
(b) Corrija a classe para que `print(t2.alunos)` mostre `[]`.

Parte 2 – Encapsulamento, propriedades e métodos especiais

4. Conta bancária [B]

Crie a classe `ContaBancaria` com `titular`, um saldo interno `_saldo` e um histórico interno `_extrato`. Requisitos:

- `depositar(valor)` e `sacar(valor)` lançam `ValueError` para valores menores ou iguais a zero; `sacar` lança `ValueError("Saldo insuficiente")` quando o valor excede o saldo;
- `saldo` é uma `@property` *somente leitura* (sem setter);
- `extrato()` devolve uma lista de tuplas `("D", valor)` ou `("S", valor)`;
- o saldo inicial, se houver, é registrado como depósito.

```
c = ContaBancaria("Maria", 100)
c.depositar(50)
c.sacar(30)
print(c.saldo)
print(c.extrato())
c.saldo = 1_000_000   # deve falhar
```

Resultado esperado: `120`, depois `[('D', 100), ('D', 50), ('S', 30)]`; a última linha lança `AttributeError`.

5. Temperatura [I]

Crie a classe `Temperatura`, que armazena o valor em Celsius e expõe duas propriedades com *getter* e *setter*: `celsius` e `fahrenheit` ($F = C \times 9/5 + 32$). O setter de `celsius` deve rejeitar valores abaixo de $-273{,}15$ com `ValueError`, e essa validação também deve valer no construtor e ao atribuir `fahrenheit`.

```
t = Temperatura(25)
print(t.fahrenheit)
t.fahrenheit = 212
print(t.celsius)
Temperatura(-300)     # deve falhar
```

Resultado esperado: `77.0`, depois `100.0`; a última linha lança `ValueError`.

6. Vetor 2D [I]

Crie a classe `Vetor2D` implementando os métodos especiais `__repr__`, `__add__`, `__sub__`, `__mul__` (por um escalar), `__eq__` e `__abs__` (módulo do vetor).

```
u, v = Vetor2D(1, 2), Vetor2D(3, 4)
print(u + v, v - u, u * 3)
print(abs(v), u == Vetor2D(1, 2))
```

Resultado esperado: `Vetor2D(4, 6) Vetor2D(2, 2) Vetor2D(3, 6)` e `5.0 True`.

7. Playlist [I]

Crie a classe `Musica` (`titulo`, `artista`, `duracao` em segundos), cujo `__str__` devolve `"titulo - artista"`, e a classe `Playlist`, que guarda músicas em uma lista interna. A `Playlist` deve:

- suportar `len(pl)`, acesso por índice `pl[i]` e iteração com `for` (dica: `__len__` e `__getitem__`);
- suportar `"titulo"in pl` (dica: `__contains__`);
- ter o método `duracao_total()`, que devolve a duração no formato `"m:ss"`.

```
pl = Playlist("Estudos")
pl.adicionar(Musica("Asa Branca", "Luiz Gonzaga", 185))
pl.adicionar(Musica("Xote das Meninas", "Luiz Gonzaga", 170))
pl.adicionar(Musica("Sina", "Djavan", 260))
print(len(pl), pl[0], "Sina" in pl, pl.duracao_total())
for m in pl:
    print(m.titulo)
```

Resultado esperado: `3 Asa Branca - Luiz Gonzaga True 10:15`, seguido dos três títulos.

Parte 3 – Herança, polimorfismo e composição

8. Veículos [B]

Crie a classe `Veiculo` (`marca`, `modelo`, `ano`) com os métodos `descricao()`, que devolve `"marca modelo (ano)"`, e `rodas()`, que devolve 0. Em seguida, crie:

- `Carro(Veiculo)`: recebe também `portas`; sobrescreve `rodas()` (4) e `descricao()`, acrescentando `- N portas"` ao texto da classe-mãe, obtido com `super()`;
- `Moto(Veiculo)`: sobrescreve apenas `rodas()` (2).

```
frota = [Carro("Fiat", "Uno", 2010, 4), Moto("Honda", "CG 160", 2022)]
for v in frota:
    print(v.descricao())
print(sum(v.rodas() for v in frota))
```

Resultado esperado: `Fiat Uno (2010) - 4 portas`, `Honda CG 160 (2022)` e `6`.

9. Formas geométricas [I]

Usando o módulo `abc`, crie a classe abstrata `Forma` com os métodos abstratos `area()` e `perimetro()` e um `__str__` concreto que devolve `"NomeDaClasse: área=X.XX"` (dica: `type(self).__name__`). Implemente as subclasses `Circulo(raio)`, `Quadrado(lado)` e `Triangulo(a, b, c)`; para a área do triângulo, use a fórmula de Heron: $A = \sqrt{s(s-a)(s-b)(s-c)}$, com s igual ao semiperímetro.

```
formas = [Circulo(1), Quadrado(2), Triangulo(3, 4, 5)]
for f in formas:
    print(f)
print("Maior:", max(formas, key=lambda f: f.area()))
Forma()                # deve falhar
```

Resultado esperado: `Circulo: área=3.14`, `Quadrado: área=4.00`, `Triangulo: área=6.00`, `Maior: Triangulo: área=6.00`; a última linha lança `TypeError`.

10. Notificações com *duck typing* [I]

Crie três classes *sem* classe-mãe em comum: `Email(endereco)`, `SMS(numero)` e `Push()`. Cada uma tem um método `enviar(msg)` que devolve uma string: `"[email para END] msg"`, `"[SMS para NUM] msg"` (com a mensagem cortada nos 20 primeiros caracteres) e `"[push] msg"`. Escreva a função `notificar_todos(canais, msg)`, que devolve a lista com os retornos de cada canal.

```
canais = [Email("ana@ufs.br"), SMS("79 99999-0000"), Push()]
for linha in notificar_todos(canais, "Prova remarcada para quinta-feira"):
    print(linha)
```

Resultado esperado:

```
[email para ana@ufs.br] Prova remarcada para quinta-feira
[SMS para 79 99999-0000] Prova remarcada para
[push] Prova remarcada para quinta-feira
```

Por que `notificar_todos` funciona sem nenhuma herança? O que aconteceria se um dos canais não tivesse o método `enviar`?

11. Pedido de lanchonete (composição) [I]

Crie a classe `Item` (`nome`, `preco_unitario`, `quantidade` com padrão 1), com o método `subtotal()`. Crie a classe `Pedido`, que *contém* uma lista de itens e oferece `adicionar(item)`, `remover(nome)` e `total(cupom=0)`, em que `cupom` é um desconto percentual (arredonde o resultado para 2 casas).

```
ped = Pedido("João")
ped.adicionar(Item("Tapioca", 8.0, 2))
ped.adicionar(Item("Suco de cajá", 6.5))
ped.adicionar(Item("Cuscuz", 10.0))
print(ped.total())
ped.remover("Cuscuz")
print(ped.total(), ped.total(cupom=10))
```

Resultado esperado: `32.5` e, depois, `22.5 20.25`. Justifique por que `Pedido` *tem* itens (composição) em vez de *ser* um item (herança).

12. Folha de salários [A]

Modele a seguinte hierarquia, com `Funcionario` abstrata e `salario()` abstrato:

- `Efetivo(nome, salario_base)`: o salário é o salário-base;
- `Gerente(nome, salario_base, bonus_pct)`, subclasse de `Efetivo`: o salário é o de `Efetivo` acrescido do bônus percentual (reaproveite o cálculo da classe-mãe com `super()`);
- `Horista(nome, valor_hora, horas)`: as horas acima de 160 são pagas com acréscimo de 50%.

```
equipe = [Efetivo("Ana", 4000), Gerente("Bruno", 6000, 20),
          Horista("Carla", 25, 170)]
print([f.salario() for f in equipe])
print(sum(f.salario() for f in equipe))
```

Resultado esperado: `[4000, 7200.0, 4375.0]` e `15575.0`.

Parte 4 – Recursos avançados

13. Alunos com `@dataclass` [I]

Use `@dataclass(order=True)` para criar a classe `Aluno` com os campos `media`, `nome` e `matricula`, de modo que a ordenação considere *apenas* a média (dica: `field(compare=False)`). Acrescente a propriedade `situacao`: `"Aprovado"` (média ≥ 7), `"Recuperação"` ($5 \leq$ média < 7) ou `"Reprovado"`.

```
alunos = [Aluno(6.5, "Davi", "2026001"), Aluno(9.0, "Bia", "2026002"),
          Aluno(4.0, "Caio", "2026003")]
print([a.nome for a in sorted(alunos, reverse=True)])
print([a.situacao for a in alunos])
print(alunos[1])
```

Resultado esperado: `['Bia', 'Davi', 'Caio']`, `['Recuperação', 'Aprovado', 'Reprovado']` e `Aluno(media=9.0, nome='Bia', matricula='2026002')`.

14. Datas com `@classmethod` e `@staticmethod` [A]

Crie a classe `Data(dia, mes, ano)` com:

- `@staticmethod eh_bissexto(ano)` e `@staticmethod eh_valida(dia, mes, ano)`;
- construtor que lança `ValueError("Data inválida")` para datas inexistentes;
- `@classmethod de_string(cls, texto)`, um construtor alternativo que recebe `"dd/mm/aaaa"`;
- `__str__` no formato `dd/mm/aaaa`, com zeros à esquerda.

```
print(Data.de_string("8/12/2026"))
print(Data.eh_bissexto(2024), Data.eh_bissexto(1900), Data.eh_bissexto(2000))
print(Data.de_string("29/02/2028"))
Data.de_string("29/02/2026")    # deve falhar
```

Resultado esperado: `08/12/2026`, `True False True`, `29/02/2028`; a última linha lança `ValueError`. Explique por que `de_string` é um método de classe, e não um método estático.

15. Desafio: sistema de biblioteca [A]

Projete as classes `Livro` (`isbn`, `titulo`, `autor`, `disponivel`), `Usuario` (`nome`, lista de empréstimos e limite de 3 livros, como atributo de classe `LIMITE`) e `Biblioteca`, que mantém o acervo em um dicionário indexado por ISBN e oferece:

- `cadastrar(livro)`;
- `emprestar(isbn, usuario)`: lança `KeyError` se o ISBN não existe e `ValueError` se o livro já está emprestado (mensagem `"'Titulo' já está emprestado"`) ou se o usuário atingiu o limite;
- `devolver(isbn, usuario)`;
- `disponiveis()`: os títulos disponíveis em ordem alfabética.

```
bib = Biblioteca()
bib.cadastrar(Livro("111", "Vidas Secas", "Graciliano Ramos"))
bib.cadastrar(Livro("222", "Dom Casmurro", "Machado de Assis"))
bib.cadastrar(Livro("333", "O Quinze", "Rachel de Queiroz"))
ana, beto = Usuario("Ana"), Usuario("Beto")
bib.emprestar("111", ana)
print(bib.disponiveis())
try:
```

```
    bib.emprestar("111", beto)
except ValueError as e:
    print(e)
bib.devolver("111", ana)
bib.emprestar("111", beto)
print([l.titulo for l in beto.emprestimos], ana.emprestimos)
```

Resultado esperado: ['Dom Casmurro', 'O Quinze'], 'Vidas Secas' já está emprestado e ['Vidas Secas'] [].